

FDCH 2 – Fraud Detection: Duplicates, Outliers, Common/Uncommon Values and DBSCAN Anomalies

Same data preparation as the Random Forest notebook (FDCH 1): repeated `Transaction_ID`s are removed (first row kept) and missing values are **imputed** (median for numbers, "Missing" for text) instead of dropping rows.

```
import os
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import chi2_contingency
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN

sns.set_style("whitegrid")
```

```
# Change the file name here if your CSV is saved under a different name
candidates = ["Fraud Detection Dataset.csv", "Fraud_Detection_Dataset.csv",
              "/content/Fraud Detection Dataset (3).csv"]
DATA_PATH = next((p for p in candidates if os.path.exists(p)), candidates[0])
df_raw = pd.read_csv(DATA_PATH)
print(df_raw.shape)
df_raw.head()
```

(51000, 12)

	Transaction_ID	User_ID	Transaction_Amount	Transaction_Type	Time_of_Transaction	Device_Used	Location	Previous_Location
0	T1	4174	1292.76	ATM Withdrawal	16.0	Tablet	San Francisco	
1	T2	4507	1554.58	ATM Withdrawal	13.0	Mobile	New York	
2	T3	1860	2395.02	ATM Withdrawal	NaN	Mobile	NaN	
3	T4	2294	100.10	Bill Payment	15.0	Desktop	Chicago	
4	T5	2130	1490.50	POS Payment	19.0	Mobile	San Francisco	

```
print(list(df_raw.columns))
```

```
['Transaction_ID', 'User_ID', 'Transaction_Amount', 'Transaction_Type', 'Time_of_Transaction', 'Device_Used', 'Location', 'Previous_Location']
```

Data cleaning

```
# Each Transaction_ID should appear once, so repeated IDs are removed (first row kept)
df = df_raw.drop_duplicates(subset="Transaction_ID", keep="first").reset_index(drop=True)
print("Rows before / after:", len(df_raw), "/", len(df))
```

Rows before / after: 51000 / 50000

```
missing = pd.DataFrame({"missing": df.isna().sum(), "percent": (df.isna().mean() * 100).round(2)})
print(missing[missing["missing"] > 0])
```

	missing	percent
Transaction_Amount	2471	4.94
Time_of_Transaction	2500	5.00
Device_Used	2433	4.87
Location	2500	5.00
Payment_Method	2422	4.84

```
print("Exact duplicate rows in raw file :", df_raw.duplicated().sum())
print("Duplicate Transaction_IDs (raw) :", df_raw["Transaction_ID"].duplicated().sum())
print("Exact duplicate rows after clean :", df.duplicated().sum())
```

Exact duplicate rows in raw file : 881
 Duplicate Transaction_IDs (raw) : 1000
 Exact duplicate rows after clean : 0

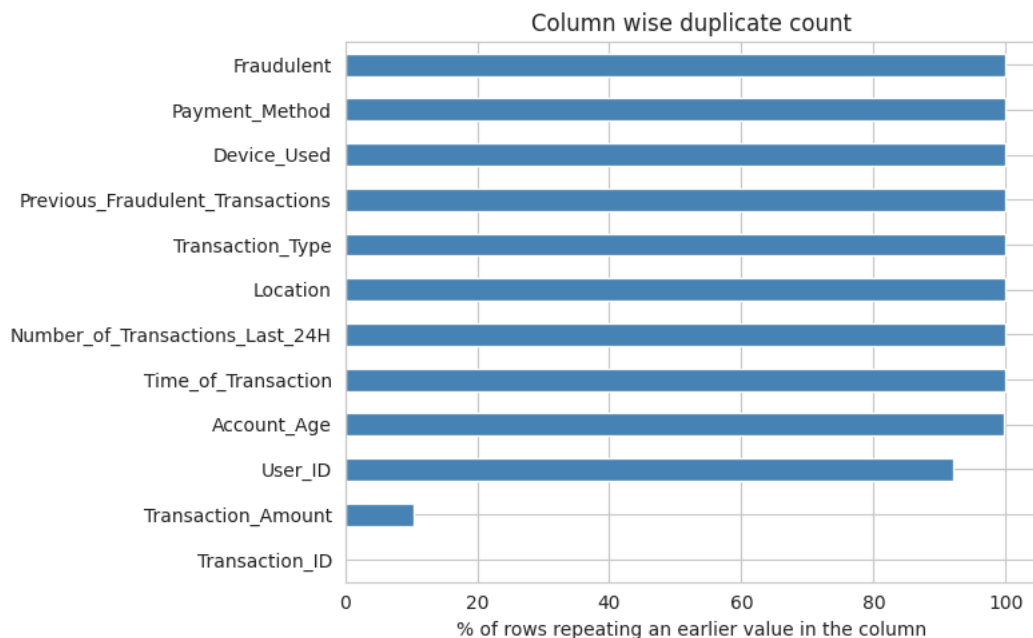
Task 1: Column wise duplicate count

For each column: how many values repeat an earlier value in that column, and how many distinct values the column has. High counts are expected for categorical columns; only `Transaction_ID` should be fully unique.

```
dup_table = pd.DataFrame({
    "duplicate_values": [df[c].duplicated().sum() for c in df.columns],
    "unique_values":    [df[c].nunique() for c in df.columns],
}, index=df.columns)
dup_table["duplicate_%"] = (dup_table["duplicate_values"] / len(df) * 100).round(2)
print(dup_table)

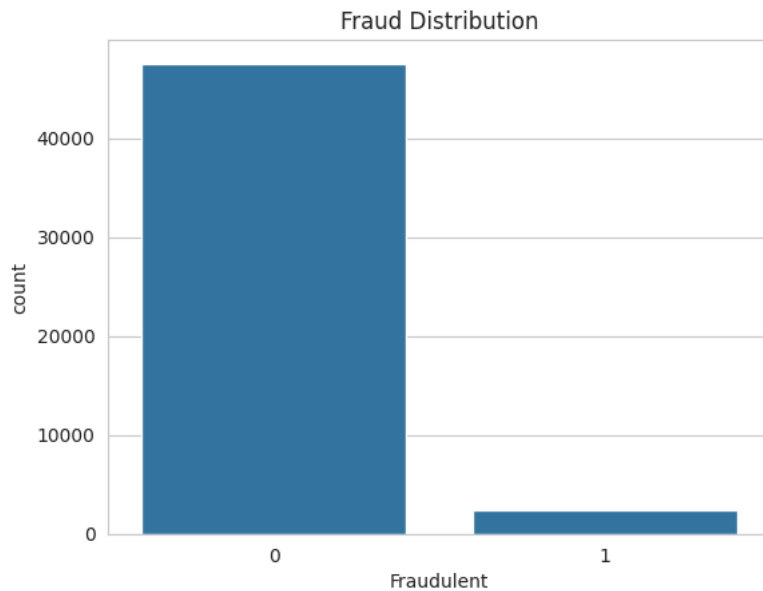
dup_table["duplicate_%"].sort_values().plot(kind="barh", figsize=(8, 5), color="steelblue")
plt.xlabel("% of rows repeating an earlier value in the column")
plt.title("Column wise duplicate count")
plt.tight_layout(); plt.show()
```

	duplicate_values	unique_values	duplicate_%
Transaction_ID	0	50000	0.00
User_ID	46000	4000	92.00
Transaction_Amount	5178	44821	10.36
Transaction_Type	49995	5	99.99
Time_of_Transaction	49975	24	99.95
Device_Used	49995	4	99.99
Location	49991	8	99.98
Previous_Fraudulent_Transactions	49995	5	99.99
Account_Age	49881	119	99.76
Number_of_Transactions_Last_24H	49986	14	99.97
Payment_Method	49994	5	99.99
Fraudulent	49998	2	100.00



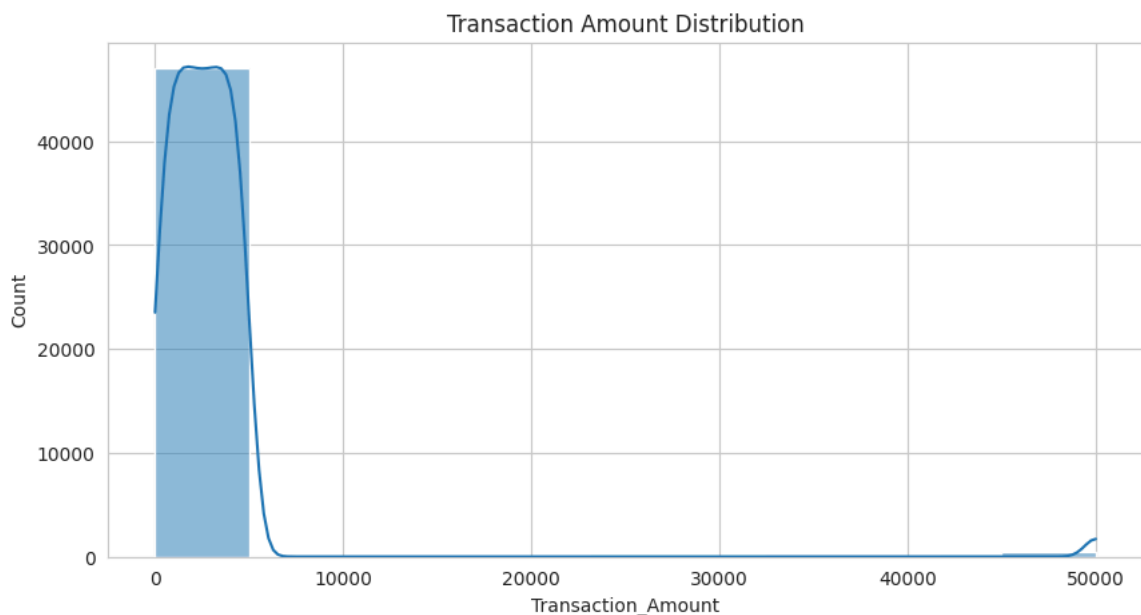
✦ Exploration

```
sns.countplot(x="Fraudulent", data=df)
plt.title("Fraud Distribution")
plt.show()
print(df["Fraudulent"].value_counts(normalize=True).round(4))
```



```
Fraudulent
0    0.9508
1    0.0492
Name: proportion, dtype: float64
```

```
plt.figure(figsize=(10, 5))
sns.histplot(df["Transaction_Amount"], bins=10, kde=True)
plt.title("Transaction Amount Distribution")
plt.show()
```



Task 2: Cluster for Payment_Method v/s Fraudulent

Clustered (grouped) bar chart of `Payment_Method` against `Fraudulent`. Missing payment methods are kept as their own "Missing" group, as in the Random Forest notebook.

```
pm = df["Payment_Method"].fillna("Missing")
order = pm.value_counts().index

fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
sns.countplot(x=pm, hue=df["Fraudulent"], order=order, ax=ax[0])
ax[0].set_title("Payment_Method vs Fraudulent (counts)")
ax[0].set_xlabel("Payment_Method"); ax[0].tick_params(axis="x", rotation=30)

rate = df.groupby(pm)["Fraudulent"].mean().reindex(order) * 100
sns.barplot(x=rate.index, y=rate.values, ax=ax[1], color="indianred")
ax[1].axhline(df["Fraudulent"].mean() * 100, color="k", ls="--", label="Overall fraud rate")
ax[1].set_title("Fraud rate by Payment_Method (%)")
ax[1].set_ylabel("% fraudulent"); ax[1].set_xlabel("Payment_Method")
ax[1].tick_params(axis="x", rotation=30); ax[1].legend()
```

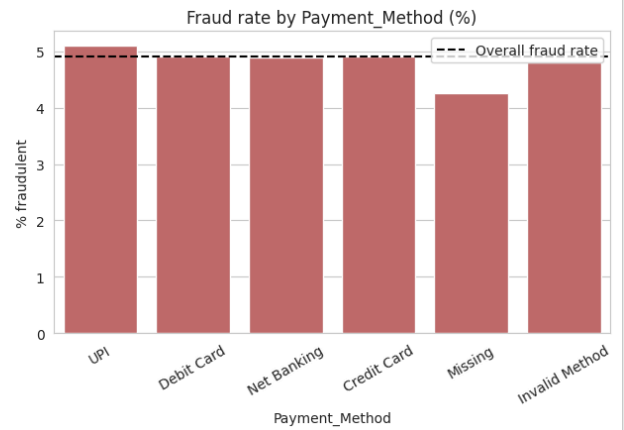
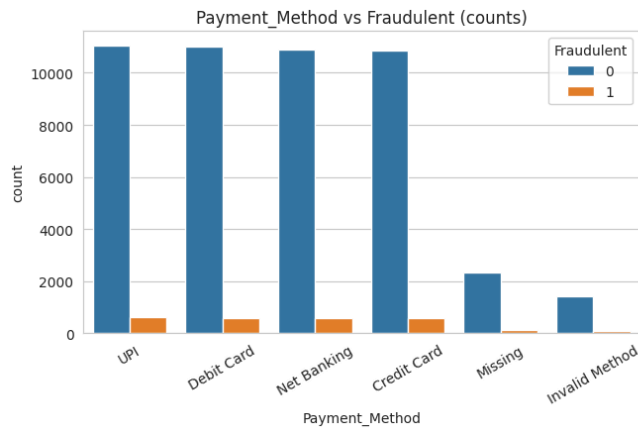
```
plt.tight_layout(); plt.show()
```

```
ct = pd.crosstab(pm, df["Fraudulent"])
```

```
print(ct)
```

```
chi, p, dof, _ = chi2_contingency(ct)
```

```
print(f"Chi-square test of independence: chi2 = {chi:.3f}, dof = {dof}, p-value = {p:.4f}")
```



Fraudulent	0	1
Payment_Method		
Credit Card	10842	561
Debit Card	11024	570
Invalid Method	1424	72
Missing	2319	103
Net Banking	10878	559
UPI	11053	595

Chi-square test of independence: chi2 = 3.250, dof = 5, p-value = 0.6615

Correlation

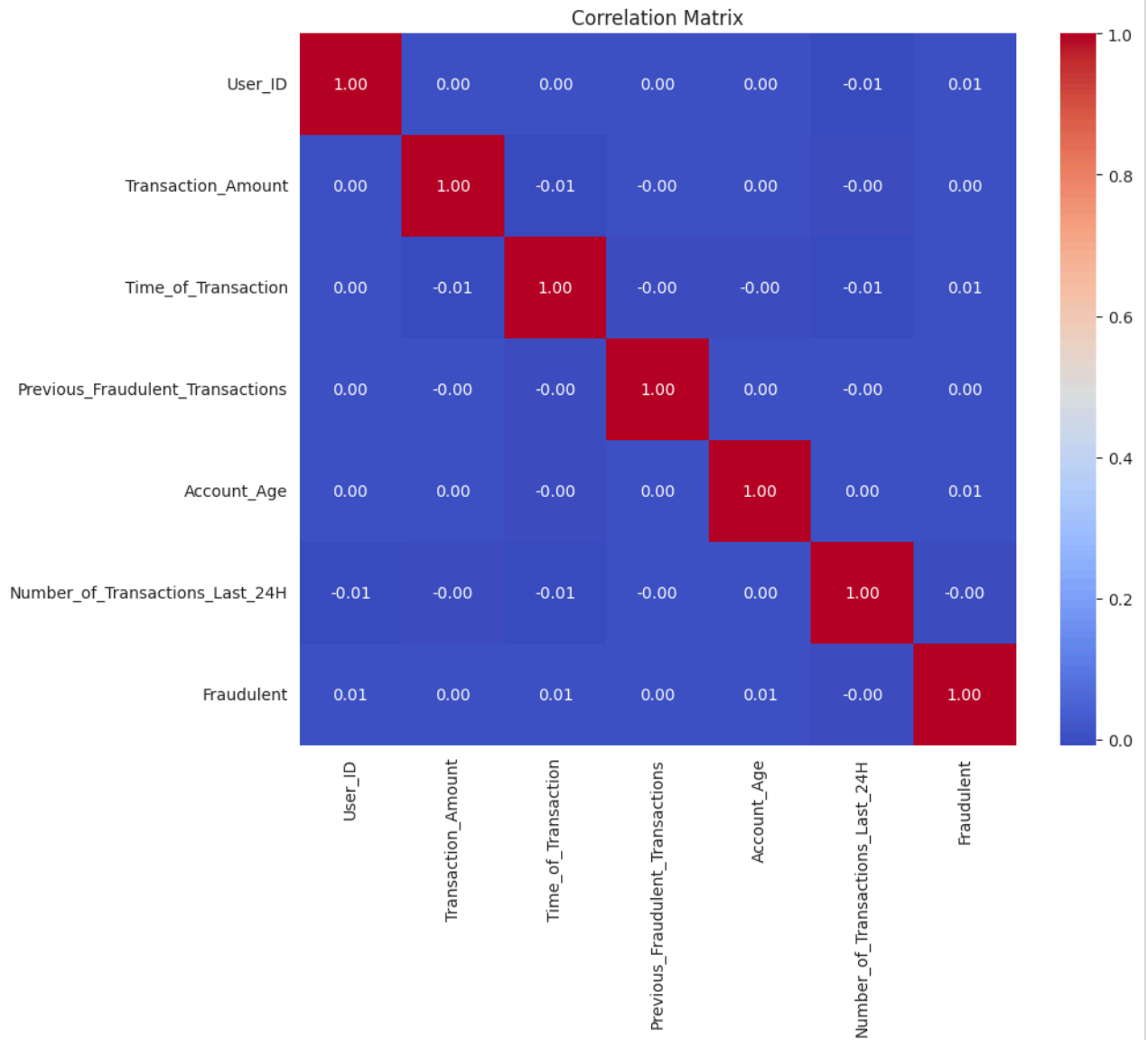
```
numeric = df.select_dtypes(include="number")
```

```
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(numeric.corr(), annot=True, fmt=".2f", cmap="coolwarm")
```

```
plt.title("Correlation Matrix")
```

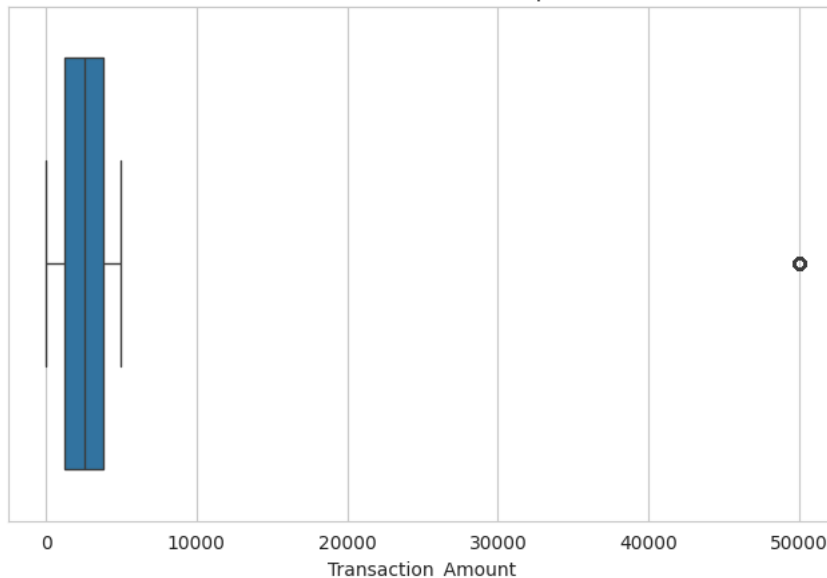
```
plt.show()
```



Outlier

```
plt.figure(figsize=(8, 5))
sns.boxplot(x=df["Transaction_Amount"])
plt.title("Transaction Amount Boxplot")
plt.show()
```

Transaction Amount Boxplot



```
def iqr_outliers(series):
    s = series.dropna()
    Q1, Q3 = s.quantile(0.25), s.quantile(0.75)
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    mask = (series < lower) | (series > upper)
    return mask, lower, upper

amt_mask, lower, upper = iqr_outliers(df["Transaction_Amount"])
outliers = df[amt_mask]

print(f"IQR bounds: lower = {lower:.2f}, upper = {upper:.2f}")
print("Outliers (shape):", outliers.shape)
print("Below lower bound:", (df["Transaction_Amount"] < lower).sum(),
      "| Above upper bound:", (df["Transaction_Amount"] > upper).sum())
print("Distinct outlier amounts:", outliers["Transaction_Amount"].nunique(),
      "-> most common:", outliers["Transaction_Amount"].mode().iloc[0])
print("Fraud rate among outliers: %.2f%% vs overall %.2f%%" %
      (outliers["Fraudulent"].mean() * 100, df["Fraudulent"].mean() * 100))
```

```
IQR bounds: lower = -2503.19, upper = 7557.37
Outliers (shape): (500, 12)
Below lower bound: 0 | Above upper bound: 500
Distinct outlier amounts: 1 -> most common: 49997.8
Fraud rate among outliers: 6.20% vs overall 4.92%
```

Task 2: Find Outliers from Time_of_Transaction, and create graph

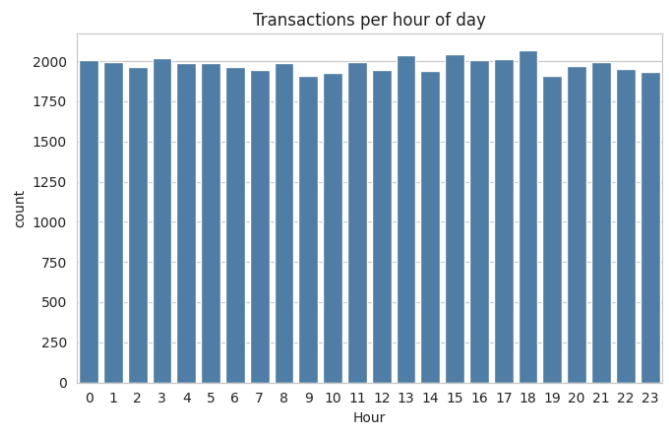
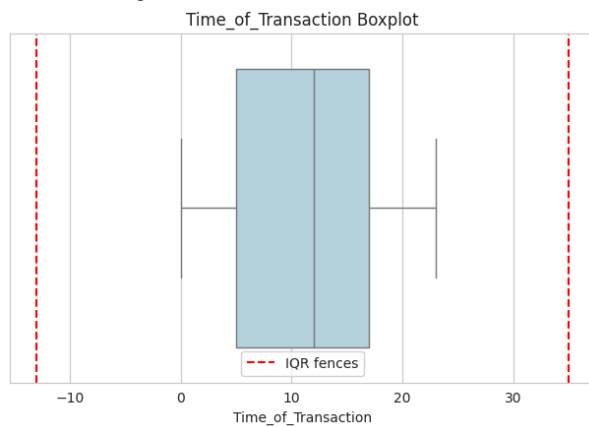
```
t_mask, t_lower, t_upper = iqr_outliers(df["Time_of_Transaction"])
t_out = df[t_mask]

print(f"IQR bounds: lower = {t_lower:.2f}, upper = {t_upper:.2f}")
print("Outliers in Time_of_Transaction:", len(t_out))
print("Observed range:", df["Time_of_Transaction"].min(), "to", df["Time_of_Transaction"].max())

fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
sns.boxplot(x=df["Time_of_Transaction"], ax=ax[0], color="lightblue")
ax[0].axvline(t_lower, color="red", ls="--"); ax[0].axvline(t_upper, color="red", ls="--", label="IQR fences")
ax[0].set_title("Time_of_Transaction Boxplot"); ax[0].legend()

sns.countplot(x=df["Time_of_Transaction"].dropna().astype(int), ax=ax[1], color="steelblue")
ax[1].set_title("Transactions per hour of day")
ax[1].set_xlabel("Hour")
plt.tight_layout(); plt.show()
```

IQR bounds: lower = -13.00, upper = 35.00
 Outliers in Time_of_Transaction: 0
 Observed range: 0.0 to 23.0



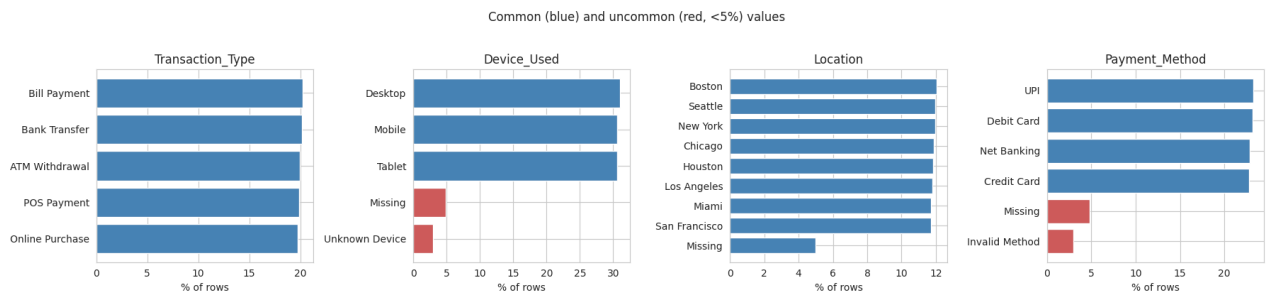
Task 3: Identify Uncommon/Common Value/s

A category is treated as **uncommon** when it makes up less than 5% of the rows (an even split over k categories would be 100/k %); the most frequent value of each column is the **common** value.

```
cat_cols = ["Transaction_Type", "Device_Used", "Location", "Payment_Method"]
rows = []
for c in cat_cols:
    vc = df[c].fillna("Missing").value_counts()
    share = (vc / len(df) * 100).round(2)
    for val, n in vc.items():
        flag = "uncommon" if share[val] < 5 else ("most common" if val == vc.index[0] else "")
        rows.append((c, val, n, share[val], flag))
cv = pd.DataFrame(rows, columns=["column", "value", "count", "percent", "flag"])
print(cv.to_string(index=False))
```

	column	value	count	percent	flag
Transaction_Type	Bill Payment	10130	20.26	most common	
Transaction_Type	Bank Transfer	10093	20.19		
Transaction_Type	ATM Withdrawal	9958	19.92		
Transaction_Type	POS Payment	9932	19.86		
Transaction_Type	Online Purchase	9887	19.77		
Device_Used	Desktop	15501	31.00	most common	
Device_Used	Mobile	15285	30.57		
Device_Used	Tablet	15281	30.56		
Device_Used	Missing	2433	4.87	uncommon	
Device_Used	Unknown Device	1500	3.00	uncommon	
Location	Boston	6027	12.05	most common	
Location	Seattle	5990	11.98		
Location	New York	5988	11.98		
Location	Chicago	5948	11.90		
Location	Houston	5918	11.84		
Location	Los Angeles	5895	11.79		
Location	Miami	5869	11.74		
Location	San Francisco	5865	11.73		
Location	Missing	2500	5.00		
Payment_Method	UPI	11648	23.30	most common	
Payment_Method	Debit Card	11594	23.19		
Payment_Method	Net Banking	11437	22.87		
Payment_Method	Credit Card	11403	22.81		
Payment_Method	Missing	2422	4.84	uncommon	
Payment_Method	Invalid Method	1496	2.99	uncommon	

```
fig, axes = plt.subplots(1, 4, figsize=(18, 4))
for a, c in zip(axes, cat_cols):
    s = df[c].fillna("Missing").value_counts(normalize=True) * 100
    colors = ["indianred" if v < 5 else "steelblue" for v in s.values]
    a.barh(s.index[:-1], s.values[:-1], color=colors[:-1])
    a.set_title(c); a.set_xlabel("% of rows")
plt.suptitle("Common (blue) and uncommon (red, <5%) values", y=1.03)
plt.tight_layout(); plt.show()
```



```
num_cols = ["Previous_Fraudulent_Transactions", "Account_Age", "Number_of_Transactions_Last_24H", "Time_of_Trans"]
for c in num_cols:
    vc = df[c].value_counts()
    print(f"{c:35s} most common: {vc.index[0]!s:>5} ({vc.iloc[0]} rows) | "
          f"least common: {vc.index[-1]!s:>5} ({vc.iloc[-1]} rows)")
```

```
amt = df["Transaction_Amount"].value_counts()
print("\nTransaction_Amount values that repeat most often:")
print(amt.head(3))
```

```
Previous_Fraudulent_Transactions    most common:      2 (10098 rows) | least common:      3 (9873 rows)
Account_Age                        most common:    106 (467 rows) | least common:    12 (376 rows)
Number_of_Transactions_Last_24H    most common:      5 (3701 rows) | least common:      9 (3449 rows)
Time_of_Transaction                most common:    18.0 (2067 rows) | least common:    19.0 (1908 rows)
```

```
Transaction_Amount values that repeat most often:
Transaction_Amount
49997.80      500
1747.31        4
246.87         3
Name: count, dtype: int64
```

DBSCAN clustering

Feature set 1: Transaction_Amount, Previous_Fraudulent_Transactions, Account_Age, Number_of_Transactions_Last_24H

Missing `Transaction_Amount` values are filled with the median (not dropped), then the features are standardised.

```
features = [
    "Transaction_Amount",
    "Previous_Fraudulent_Transactions",
    "Account_Age",
    "Number_of_Transactions_Last_24H",
]
```

```
X = df[features].fillna(df[features].median())

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
dbscan = DBSCAN(eps=0.8, min_samples=15)
labels = dbscan.fit_predict(X_scaled)
```

```
df["Cluster"] = labels
df["Cluster"].value_counts()
```

```
Cluster
0      49500
1        495
-1         5
Name: count, dtype: int64
```

Task 3 (continued): cluster 0 is the normal bulk of transactions, the other clusters are small dense groups, and label -1 marks points that belong to no cluster.

```
cluster_summary = df.groupby("Cluster").agg(
    rows=("Cluster", "size"),
    fraud_rate_pct=("Fraudulent", lambda s: round(s.mean() * 100, 2)),
    mean_amount=("Transaction_Amount", "mean"),
    mean_prev_fraud=("Previous_Fraudulent_Transactions", "mean"),
).round(2)
cluster_summary
```

	rows	fraud_rate_pct	mean_amount	mean_prev_fraud
Cluster				
-1	5	20.00	49997.80	0.40
0	49500	4.91	2497.45	2.00
1	495	6.06	49997.80	1.99

Task 4: Identify Anomalies with -1 (for different feature set)

First for feature set 1 above, then for a different feature set (Transaction_Amount, Time_of_Transaction, Account_Age, Number_of_Transactions_Last_24H).

```
suspicious = df[df["Cluster"] == -1]

print("Anomalies in feature set 1:", len(suspicious))
suspicious.head()
```

Anomalies in feature set 1: 5

	Transaction_ID	User_ID	Transaction_Amount	Transaction_Type	Time_of_Transaction	Device_Used	Location	P
15441	T15442	4030	49997.8	Bank Transfer	20.0	Desktop	Seattle	
19824	T19825	2206	49997.8	Online Purchase	14.0	Mobile	Miami	
28247	T28248	1065	49997.8	Online Purchase	11.0	Unknown Device	Los Angeles	
34603	T34604	1646	49997.8	Bill Payment	16.0	Mobile	Seattle	
37997	T37998	2515	49997.8	POS Payment	16.0	Tablet	Boston	

```
features2 = [
    "Transaction_Amount",
    "Time_of_Transaction",
    "Account_Age",
    "Number_of_Transactions_Last_24H",
]

X2 = df[features2].fillna(df[features2].median())
X2_scaled = StandardScaler().fit_transform(X2)

df["Cluster2"] = DBSCAN(eps=0.5, min_samples=15).fit_predict(X2_scaled)
print(df["Cluster2"].value_counts())

suspicious2 = df[df["Cluster2"] == -1]
print("\nAnomalies in feature set 2:", len(suspicious2))
suspicious2.head()
```

```
Cluster2
0    49500
-1     500
Name: count, dtype: int64
```

Anomalies in feature set 2: 500

	Transaction_ID	User_ID	Transaction_Amount	Transaction_Type	Time_of_Transaction	Device_Used	Location	Pre
166	T167	1502	49997.8	POS Payment	3.0	Desktop	Miami	
236	T237	2529	49997.8	ATM Withdrawal	3.0	Tablet	NaN	
245	T246	2306	49997.8	Bill Payment	1.0	Desktop	New York	
298	T299	2153	49997.8	Online Purchase	18.0	Tablet	San Francisco	
317	T318	4681	49997.8	Bill Payment	0.0	Mobile	Houston	

```
summary = pd.DataFrame({
    "feature set": ["1: Amount, Prev_Fraud, Account_Age, Txn_24H", "2: Amount, Time, Account_Age, Txn_24H"],
    "anomalies (-1)": [len(suspicious), len(suspicious2)],
    "fraud rate in anomalies %": [round(suspicious["Fraudulent"].mean() * 100, 2) if len(suspicious) else np.nan,
    round(suspicious2["Fraudulent"].mean() * 100, 2) if len(suspicious2) else np.nan],
    "overall fraud rate %": [round(df["Fraudulent"].mean() * 100, 2)] * 2,
})
print(summary.to_string(index=False))
print("\nAnomalies of set 2 with the repeated amount", amt.index[0], ":",
      (suspicious2["Transaction_Amount"] == amt.index[0]).sum(), "of", len(suspicious2))
```

	feature set	anomalies (-1)	fraud rate in anomalies %	overall fraud rate %
1: Amount, Prev_Fraud, Account_Age, Txn_24H		5	20.0	4.92
2: Amount, Time, Account_Age, Txn_24H		500	6.2	4.92

Anomalies of set 2 with the repeated amount 49997.8 : 500 of 500

Task 5: Create Graphs for Anomalies

Task 5: Cluster Graph (any two attributes)

```
x_attr = "Transaction_Amount"
y_attr = "Previous_Fraudulent_Transactions"
```

```
non_noise = df[df["Cluster"] != -1]
noise = df[df["Cluster"] == -1]

sample_size = min(3000, len(non_noise))
plot_sample = non_noise.sample(n=sample_size, random_state=42)

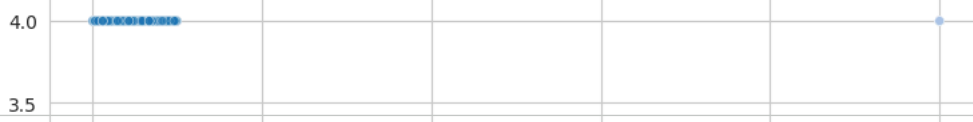
plt.figure(figsize=(9, 7))

sns.scatterplot(
    data=plot_sample,
    x=x_attr, y=y_attr,
    hue="Cluster",
    palette="tab20",
    alpha=0.6, s=25,
    legend=False
)

plt.scatter(
    noise[x_attr], noise[y_attr],
    c="red", marker="o", s=70,
    label=f"Anomaly (Cluster = -1), n={len(noise)}"
)

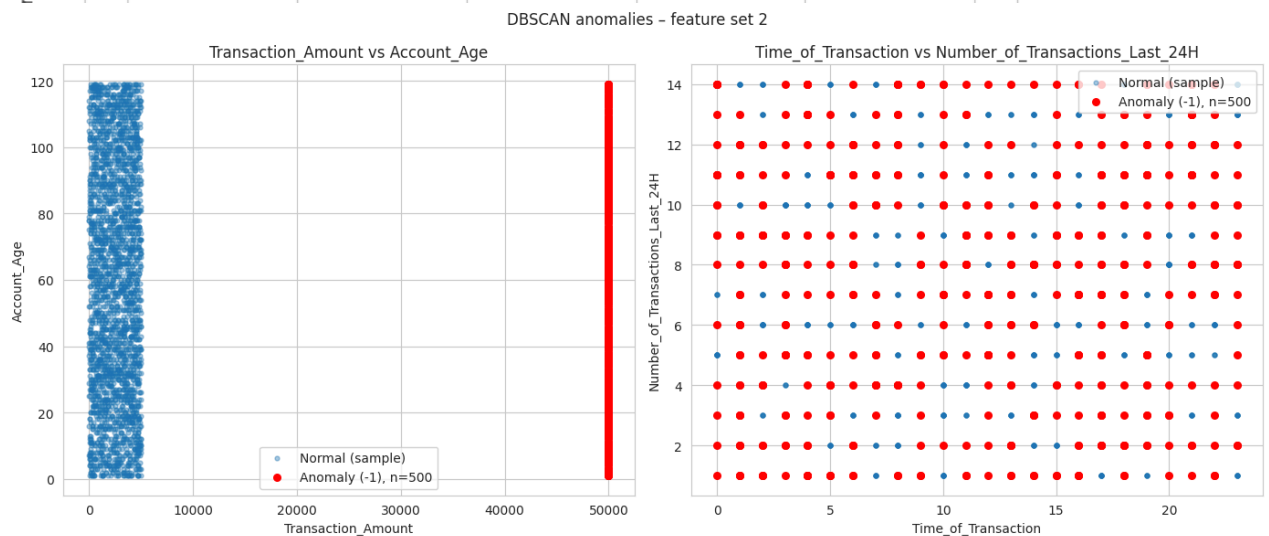
plt.xlabel(x_attr)
plt.ylabel(y_attr)
plt.title(f"DBSCAN Clusters - {x_attr} vs {y_attr}")
plt.legend()
plt.show()
```

DBSCAN Clusters — Transaction_Amount vs Previous_Fraudulent_Transactions



```
# Feature set 2: the anomalies are the transactions with the repeated, extreme amount
non_noise2 = df[df["Cluster2"] != -1]
noise2 = df[df["Cluster2"] == -1]
plot_sample2 = non_noise2.sample(n=min(3000, len(non_noise2)), random_state=42)

fig, ax = plt.subplots(1, 2, figsize=(14, 6))
for a, (xa, ya) in zip(ax, [("Transaction_Amount", "Account_Age"),
                             ("Time_of_Transaction", "Number_of_Transactions_Last_24H")]):
    a.scatter(plot_sample2[xa], plot_sample2[ya], s=12, alpha=0.4, label="Normal (sample)")
    a.scatter(noise2[xa], noise2[ya], c="red", s=30, label=f"Anomaly (-1), n={len(noise2)}")
    a.set_xlabel(xa); a.set_ylabel(ya); a.set_title(f"{xa} vs {ya}"); a.legend()
plt.suptitle("DBSCAN anomalies – feature set 2")
plt.tight_layout(); plt.show()
```



```
# Are anomalies more often fraudulent than normal transactions?
comp = pd.DataFrame({
    "Normal": [non_noise["Fraudulent"].mean() * 100, non_noise2["Fraudulent"].mean() * 100],
    "Anomaly (-1)": [noise["Fraudulent"].mean() * 100, noise2["Fraudulent"].mean() * 100],
}, index=["Feature set 1", "Feature set 2"])
comp.plot(kind="bar", figsize=(7, 4), rot=0)
plt.ylabel("% fraudulent"); plt.title("Fraud rate: normal vs anomaly")
plt.tight_layout(); plt.show()
print(comp.round(2))
```

